

Akshith Macharla

Chico, CA | amacharla@csuchico.edu | +1 530-965-7864 | github.com/amacharla15 | linkedin.com/in/akshith-macharla-

Education

California State University, Chico

Master of Science in Computer Science – GPA 3.95/4.00

Coursework: Operating Systems, Data Structures and Algorithms, Software Design, Machine Learning, Computer Vision

Chico, CA

Expected Dec 2026

Chaitanya Bharathi Institute of Technology

Bachelor of Engineering in Electrical and Electronics Engineering

Hyderabad, India

2017 – 2021

Technical Skills

- **Languages:** C++, Python, CUDA, C, Java, SQL
- **LLM Inference:** vLLM, FlashInfer, paged KV cache, tensor parallelism, FP8 / NVFP4 / INT8 quantization, llm-compressor
- **GPU & Kernels:** CUDA, CUTLASS, Nsight Systems, NCCL, attention kernels; NVIDIA A100, H100, L4, RTX 4090
- **Frameworks & Tools:** PyTorch, torch.compile, FX / PT2E, ExecuTorch, LLVM, Linux, Docker, Git, CMake, pytest, AWS

Open-Source Contributions

PyTorch ExecuTorch – quantized attention lowering for Arm Cortex-M | *Python, FX, PT2E*

Merged | PR #22892

- Wrote an SDPA decomposition pass that applies the full attention scale once after the QK matmul, not as $d^{-1/4}$ on both Q and K, so int8 scales stay proportional and fold into requantization; added end-to-end SDPA tests.

NVIDIA CUTLASS – FP8 blockwise GEMM split-K correctness fix | *C++, CUDA*

In review | PR #3615

- Fixed serial split-K blockwise-scaled GEMM silently applying slice 0's scale factors to every K-slice (output **1280 instead of 10,880** while every status check returned `kSuccess`) by threading the slice's K-tile offset through four mainloop layers to the scale lookup.
- Extended SM89 tests from K=128, a single scale block that could never expose the bug, to K=512 with 4 scale blocks and split-K 1/2/4 under both threadblock swizzles; validated on an RTX 4090.

vLLM llm-compressor – fused-layer quantization mappings | *Python, PyTorch, NVFP4*

In review | PR #3224

- Unified scattered fused-layer mappings (QKV, gate/up, MLA, Kimi KDA, DeepSeek V4) into one resolver so layers vLLM packs together share one scale; on H100 with Kimi-K3, attention groups with mismatched NVFP4 scales fell from 8/8 to 0/8.

Research & Projects

POD-Attention Reproduction – fused prefill + decode attention (ASPLOS '25) | *CUDA, FlashInfer*

Research, in progress

- Reproducing POD-Attention on A100 and extending it to H100 across 54 prefill/decode mixes, against sequential and concurrent-stream baselines; validated FlashInfer POD on H100 against standalone prefill and decode kernels (max abs error 1.2×10^{-4}).

CUDA Transformer Kernels | *CUDA, C++, PyTorch, Nsight Systems*

github.com/amacharla15/CUDAkernels

- Hand-wrote CUDA kernels for shared-memory tiled GEMM, softmax, RMSNorm, multi-head attention, and FlashAttention-style tiled attention with online softmax, each checked against PyTorch and profiled with Nsight Systems.

Multi-GPU LLM Serving Benchmarks | *vLLM, PyTorch Distributed, NCCL*

2x A100-SXM4-80GB

- Benchmarked vLLM serving of Qwen2.5-1.5B-Instruct with tensor parallelism 2, measuring TTFT, TPOT, and throughput across prompt lengths and concurrency; measured NCCL all-reduce at 172.6 GB/s and broadcast at 252.9 GB/s (256 MB).

IRGuard – LLM agent for safe LLVM GPU optimization | *C++, LLVM, PTX, Python*

github.com/amacharla15/ML-accelerators

- LLVM pass plugin applying fast-math flags to CUDA kernel IR via NVPTX, plus an fp64 verifier on an L4 that returns concrete counterexamples; with them, an LLM agent fixed the exposed failure first in 10/10 runs vs. 3/10.

Parallel BPE Tokenizer | *C++, multithreading*

github.com/amacharla15/Parallel-BPE-Tokenizer

- GPT-2 byte-level BPE in C++ with a fixed thread pool and per-thread merge caches: $5.0 \times$ scaling from 1 to 8 threads, 2.11M tokens/s with caching, 91.6% of tiktoken's throughput.

Experience

California State University, Chico

Research Assistant, 3D Computer Vision

Chico, CA

Feb 2026 – Jun 2026

- Built C++/OpenCV tools and Linux pipelines for RealSense RGB-D data: depth/color alignment, filtering, road-plane fitting.

Cognizant Technology Solutions

Programmer Analyst / Associate Software Engineer

Hyderabad, India

Dec 2021 – Aug 2024

- Built and maintained Java/Spring Boot and PostgreSQL backend services for insurance policy lookup, claims intake, workflow status, and audit history, used by 10,000+ active users.
- Cut p95 API latency from ~250 ms to ~200 ms by removing redundant reads and optimizing Hibernate/SQL data access.
- Triaged 100+ production incidents (API failures, database inconsistencies, performance regressions) using Dynatrace and Linux logs.

Certifications & Activities

- **AWS Certified Developer – Associate** | **LeetCode:** 1600+ rating | **LeetGPU:** leetgpu.com/BlazingOracle84a96
- **Cal Hacks:** Built Chaos Reviewer, an AI code-review agent (Fetch.ai track) that logged ~7.2K agent interactions